Vehicle Rental Management System – Design Report

Object-Oriented Programming Assignment

Student: Lojain Yadak

1. Introduction

This report explains the object-oriented design of a console-based Java application for a vehicle rental company that rents cars, motorcycles and trucks. The system manages vehicles, customers, rentals, returns, maintenance and revenue. The source code is organized by responsibility: `exception` (custom exceptions), `model` (domain classes and the `Maintainable` interface), `service` (`RentalSystem`, which holds all business logic) and `main` (`Main`, which only handles console input and output).

2. Abstraction and the abstract class

Abstraction means exposing *what* an object does while hiding *how* it does it. `Vehicle` is an abstract class. It holds the state and behavior shared by every vehicle: ID, brand, model, year, daily price, availability and a maintenance flag, together with validation and the helper `baseCost(days)`. It also declares the abstract method `calculateRentalCost(int days, boolean optionalService)`.

A generic "vehicle" has no meaningful cost rule on its own, so `Vehicle` cannot be instantiated, and every concrete subclass is forced to supply the rule. The rest of the system works with `Vehicle` references and never needs to know which concrete type it holds.

3. Abstract classes versus interfaces

An abstract class models an *is-a* relationship and can contain state, constructors and implemented methods. `Vehicle` uses this to share fields, validation and `baseCost()` between `Car`, `Motorcycle` and `Truck`.

An interface defines a *capability contract* without state. `Maintainable` (`performMaintenance()` and `needsMaintenance()`) describes anything that can be serviced, independent of its place in the class hierarchy. Java allows a class to extend only one superclass but to implement many interfaces, so `Car`, `Motorcycle` and `Truck` extend `Vehicle` and also implement `Maintainable`. If a new kind of asset (for example a trailer) needed servicing, it could implement `Maintainable` without inheriting from `Vehicle`.

In short: the abstract class answers "what is it, and what do all vehicles share?", while the interface answers "what can it do?".

4. Polymorphism – two forms

Inheritance-based (runtime) polymorphism. `RentalSystem` stores vehicles in an `ArrayList<Vehicle>`. When a `Rental` is created it calls `vehicle.calculateRentalCost(days, service)`, and the JVM chooses the implementation of the actual object at runtime:

- `Car`: base cost plus $15 per day if insurance is selected.
- `Motorcycle`: base cost plus $5 per day if a helmet is selected.
- `Truck`: base cost plus a fixed $50 service fee, once per rental.

No if/else or switch on the vehicle type exists anywhere in the program. The same mechanism is used for `displayInfo()`, which `Main.printVehicles()` calls on `Vehicle` references.

Interface-based polymorphism. `RentalSystem.getMaintainableFleet()` returns an `ArrayList<Maintainable>`. Methods such as `countNeedingMaintenance()` and `performMaintenance()` use only `Maintainable` references, so `m.performMaintenance()` runs the car's, the motorcycle's or the truck's own routine (oil and brakes, chain lubrication, axle inspection) without knowing the concrete class.

5. Method overloading

Overloading means several methods share a name but have different parameter lists, and the compiler selects the version at compile time.

- `RentalSystem.searchVehicles(String keyword)` searches by brand or model, while `searchVehicles(double minPrice, double maxPrice)` searches by daily price range.

- Constructors are overloaded: `Car` has a full constructor (seats and transmission) and a shorter one that calls `this(...)` to create a standard 5-seat automatic car. `Motorcycle` offers the same with a default helmet option.

6. Method overriding

Overriding means a subclass provides its own implementation of an inherited method with the same signature, resolved at runtime. Each subclass overrides `calculateRentalCost()` (mandatory, because it is abstract) and `displayInfo()`, which first calls `super.displayInfo()` and then prints its own attributes. `Car` and `Motorcycle` also override `getOptionalServiceName()`, whose default in `Vehicle` returns `null`. Each class implements the two `Maintainable` methods with its own behavior.

Overloading and overriding differ in when and how they are resolved: overloading is chosen by the compiler from the argument types (same class, same name, different parameters), whereas overriding is chosen by the JVM from the object's real type (subclass, same signature).

7. Other object-oriented concepts used

- **Encapsulation:** all fields are private. Changes go through validated setters; for example `setDailyPrice` rejects values of zero or less, and `Customer` validates phone and email formats.
- **this and super:** `this(...)` chains constructors, and `super(...)` builds the base part of each subclass object.
- **static and final:** `Rental.rentalCounter` is a static counter that produces unique IDs (R001, R002, ...). The fees `INSURANCE_FEE_PER_DAY`, `HELMET_FEE_PER_DAY` and `SERVICE_FEE` are `public static final` constants.
- **Association:** a `Rental` references one `Customer` and one `Vehicle`, and each `Customer` keeps its rentals in an `ArrayList`.
- **Exceptions and validation:** the custom checked exceptions `InvalidInputException` and `VehicleNotAvailableException` enforce the business rules: duplicate IDs, non-positive rental days, unknown customers or vehicles, renting a rented or flagged vehicle, and returning a rental twice. `Main` catches them and its input helpers re-prompt on bad input, so the program never terminates unexpectedly.

8. Maintenance design

Maintenance is tracked independently from availability. A vehicle is rentable only if it is available *and* not flagged for maintenance (`isRentable()`). Staff can flag a vehicle at any time, but maintenance cannot be performed while the vehicle has an active rental. Performing maintenance clears the flag, after which the vehicle can be rented again.

9. Separation of responsibilities

`Main` reads input and prints results only. All business rules live in `RentalSystem` and in the model classes, which keeps the logic reusable and testable. Revenue is computed from completed rentals only.

10. Conclusion

The design combines an abstract base class for shared state and the cost contract, an interface for the cross-cutting maintenance capability, and two kinds of polymorphism that remove any need for type checks. Encapsulation, validation and custom exceptions keep the data consistent, and the separation between `Main` and `RentalSystem` keeps the code organized and easy to extend with new vehicle types.